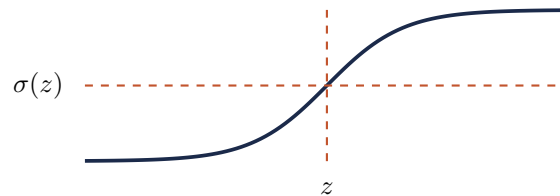


Classification & Logistic Regression

A Practical and Mathematical Learning Guide for Python and Kaggle

This guide takes you from the statistical foundations of classification to a complete, reproducible Kaggle-style logistic regression project — covering the mathematics behind the sigmoid function, model training, evaluation metrics, regularization, and practical Scikit-learn implementation.



Intermediate Level

From Statistical Foundations to Applied Machine Learning

August 30, 2026

Contents

1	Understanding a Classification Dataset	3
1.1	Machine Learning and Supervised Learning	3
1.2	Classification vs Regression	3
1.3	Binary Classification	3
1.4	Multiclass Classification	4
2	Dataset Inspection with Pandas	4
2.1	Loading and Previewing Data	4
2.2	Handling Missing and Unwanted Data	5
2.3	Pandas Inspection Methods at a Glance	5
3	Statistical Foundation and Exploratory Understanding	6
3.1	Mean	6
3.2	Variance	6
3.3	Standard Deviation	6
3.4	Covariance	6
3.5	Correlation	7
3.6	Python Implementation	7
4	Why Classification Needs Logistic Regression	7
4.1	Why Not Just Use Linear Regression?	8
4.2	Why Not Just Clip the Output?	8
4.3	Introducing Logistic Regression	8
5	Logistic Regression Mathematics	8
5.1	The Linear Score	9
5.2	The Sigmoid Function	9
5.3	Sigmoid Behavior	9
5.4	From Probability to Class	10
6	Log-Odds, Coefficients, and How the Model Learns	10
6.1	Odds and Log-Odds	10
6.2	The Key Relationship	10
6.3	Interpreting Coefficients	11
6.4	Odds Ratio	11
6.5	How Logistic Regression Learns	11
7	Implementing Logistic Regression in Python	12
7.1	Line-by-Line Explanation	13
7.2	Predict vs Predict-Probab	14
8	Binary and Multiclass Logistic Regression	14
8.1	Binary Logistic Regression	14
8.2	Multiclass Classification	14
8.3	One-vs-Rest vs Multinomial	15
9	Evaluating Classification Models	15
9.1	The Confusion Matrix	15
9.2	Accuracy	16
9.3	Precision	16
9.4	Recall	16
9.5	Specificity	16

9.6	F1 Score	16
9.7	Precision vs Recall: Choosing What Matters	17
9.8	ROC-AUC	17
9.9	Evaluating Models in Python	17
10	Class Imbalance, Overfitting, Regularization and Preprocessing	18
10.1	Class Imbalance	18
10.2	Overfitting and Underfitting	18
10.3	Regularization	19
10.4	Feature Scaling	20
10.5	Data Leakage	20
11	Complete Kaggle-Style Classification Project	21
11.1	The Project	21
11.2	The Workflow	21
11.3	Step 1 – Load and Inspect	21
11.4	Step 2 – Define Features and Target, Then Split	22
11.5	Step 3 – Preprocessing Pipeline	22
11.6	Step 4 – Full Model Pipeline and Training	22
11.7	Step 5 – Predict and Evaluate	23
11.8	Step 6 – Interpret Errors and Check Overfitting	23
11.9	Step 7 – Tune Threshold if Necessary	23
11.10	Step 8 – Predict for a New Customer	23
12	Binary vs Multiclass + Logistic Regression Decision Guide	24
12.1	Binary vs Multiclass	24
12.2	Logistic Regression vs Linear Regression	24
12.3	Evaluation Metric Guide	25
13	Final Formula Sheet	25
13.1	Descriptive Statistics	25
13.2	Logistic Regression Core	25
13.3	Regularization	26
13.4	Classification Evaluation Metrics	26
14	Final Python Cheat Sheet	26
15	Final Mental Model	27

1 Understanding a Classification Dataset

1.1 Machine Learning and Supervised Learning

Machine learning is the practice of building systems that learn patterns from data rather than following explicitly programmed rules. Instead of writing `if` statements for every possible situation, we show the system many examples and let it infer the underlying pattern.

Supervised learning is the branch of machine learning where every training example comes with a known answer. The model's job is to learn the mapping between inputs and answers well enough to predict the answer for new, unseen inputs.

In supervised learning we use the following vocabulary:

Core Vocabulary

- **Features (“X”)** — the measurable input variables describing each example (e.g. hours studied, age, income).
- **Target / label (“y”)** — the value we are trying to predict (e.g. pass/fail, churn/no churn).
- **Training data** — the portion of the dataset used to let the model learn patterns.
- **Testing data** — a separate portion, held back from training, used to check how well the model generalizes to new data.
- **Prediction** — the model's output when given features it has not seen the label for.

A useful mental model: the model is a function f that we want to learn, such that $f(X) \approx y$. Supervised learning uses labeled examples to search for a good approximation of f .

1.2 Classification vs Regression

Supervised learning problems fall into two broad categories, depending on the type of target variable.

Aspect	Regression	Classification
Output type	Continuous numerical value	Discrete class label
Question asked	“How much?” / “How many?”	“Which category?”
Example	Predicting a house price (\$245,000)	Predicting spam vs not spam

If the target can take any numeric value along a continuous range (price, temperature, salary), it is a **regression** problem. If the target belongs to a fixed, finite set of categories (yes/no, species, risk level), it is a **classification** problem. This chapter — and this entire guide — focuses on classification.

1.3 Binary Classification

Binary classification problems have exactly two possible classes, usually encoded as 0 and 1.

Problem	Class 0	Class 1
Email filtering	Not Spam	Spam
Transaction monitoring	Not Fraud	Fraud
Exam outcome	Fail	Pass
Subscription behavior	No Churn	Churn

1.4 Multiclass Classification

Multiclass classification problems involve more than two possible classes. Each example still belongs to exactly one class, but there are three or more categories to choose from.

Problem	Possible Classes
Animal image recognition	Cat / Dog / Horse
Flower species identification	Iris Setosa / Versicolor / Virginica
Risk categorization	Low / Medium / High

Important distinction: “Binary” and “multiclass” describe the *structure of the problem* — how many classes exist. **Logistic Regression** is one specific *algorithm* that can be adapted to solve either type of problem. Do not confuse the type of problem with the algorithm used to solve it; many algorithms (decision trees, support vector machines, neural networks) can also handle both binary and multiclass classification.

2 Dataset Inspection with Pandas

Before any modeling begins, a Kaggle project always starts with understanding the raw data: how many rows and columns exist, what type of information each column holds, whether values are missing, and whether anything looks statistically unusual. Skipping this step is one of the most common reasons beginners build broken models.

2.1 Loading and Previewing Data

```
import pandas as pd

df = pd.read_csv("dataset.csv")

df.head()
df.info()
df.describe()
```

head()

`df.head()` displays the first five rows of the dataset by default. This gives an immediate, human-readable sense of what the columns actually contain — their names, the kind of values stored in them, and how they are formatted. It is almost always the very first command run after loading a dataset.

info()

`df.info()` summarizes the structure of the entire DataFrame. It reports:

- The total number of **rows** in the dataset.
- The list of **columns**.
- The **data type** of each column (e.g. `int64`, `float64`, `object`).
- The **non-null count** for each column, which reveals **missing values** at a glance.

Data types matter because they determine what preprocessing is required. A column stored as `object` (text) that actually represents a category, such as "Yes"/"No", will need to be encoded into a numeric form before a logistic regression model can use it. A numeric column that was accidentally loaded as text (for example, due to stray characters) will silently fail to behave as expected in

calculations.

`describe()`

`df.describe()` produces summary statistics for the numerical columns:

- **count** — how many non-missing values exist in the column.
- **mean** — the average value.
- **std** — the standard deviation (spread around the mean).
- **min** and **max** — the smallest and largest observed values.
- **percentiles** (25%, 50%, 75%) — values that split the data into ordered fractions.

This single command can reveal statistical abnormalities quickly: a minimum age of -5 , a maximum income of 999,999,999, or a standard deviation of zero (meaning a column has no useful information at all) are all signs that the data needs cleaning before it is used.

2.2 Handling Missing and Unwanted Data

Four Pandas methods are used constantly during this inspection stage:

```
df.isnull()
df.drop()
df.dropna()
df.fillna()
```

- `df.isnull()` returns a same-shaped DataFrame of `True/False` values, flagging exactly where data is missing. It is often combined with `.sum()` to count missing values per column.
- `df.drop()` removes specified rows or columns — useful when a column is irrelevant, is a duplicate of another feature, or contains an identifier (like a customer ID) that carries no predictive signal.
- `df.dropna()` removes rows (or columns) that contain missing values. This is simple but can discard useful data if used carelessly on a dataset with many small gaps.
- `df.fillna()` replaces missing values with a chosen substitute, such as the column mean, median, or a constant. This preserves the row while addressing the gap.

The choice between dropping and filling depends on how much data is missing and why. A column that is 90% empty is often better dropped; a column missing only a handful of values is often better filled.

2.3 Pandas Inspection Methods at a Glance

Method	Purpose
<code>df.head()</code>	Preview the first rows of the dataset
<code>df.info()</code>	Inspect structure: rows, columns, types, missing values
<code>df.describe()</code>	Statistical summary of numeric columns
<code>df.isnull()</code>	Flag missing values
<code>df.drop()</code>	Remove specified rows or columns
<code>df.dropna()</code>	Remove rows/columns containing missing values
<code>df.fillna()</code>	Replace missing values with a chosen value

3 Statistical Foundation and Exploratory Understanding

Before fitting any model, it helps to understand the numerical behavior of the data and how features relate to each other. The following statistics are the standard toolkit for that exploration.

3.1 Mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The mean is the arithmetic average of a set of values: sum everything and divide by the number of observations, n . It gives a single number representing the “center” of the data. The mean is sensitive to extreme values (outliers) — a single very large or very small value can pull the mean noticeably away from where most of the data actually sits.

3.2 Variance

$$Var(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Variance measures how spread out the values are around the mean. Each value’s distance from the mean is squared (so negative and positive deviations don’t cancel out) and then averaged. A variance near zero means the values are tightly clustered around the mean; a large variance means the values are widely scattered.

3.3 Standard Deviation

$$\sigma = \sqrt{Var(X)}$$

Standard deviation is simply the square root of variance. Because variance is expressed in *squared* units (e.g. squared dollars), it is hard to interpret intuitively. Taking the square root brings the measure of spread back into the original units (e.g. dollars), making it far easier to reason about — for example, “most values fall within one standard deviation of the mean.”

3.4 Covariance

$$Cov(X, Y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Covariance measures whether two variables tend to move together.

- **Positive covariance** — when X is above its mean, Y tends to be above its mean too (they move in the same direction).
- **Negative covariance** — when X is above its mean, Y tends to be below its mean (they move in opposite directions).
- **Near-zero covariance** — little to no linear relationship between the two variables.

The main limitation of covariance is that its magnitude depends on the units and scale of the variables involved, which makes it difficult to compare the strength of relationships across different variable pairs. A covariance of 500 between income and spending might reflect a weaker relationship than a covariance of 2 between two variables measured on a 0–1 scale.

3.5 Correlation

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Correlation solves covariance's scaling problem by dividing by the standard deviations of both variables, producing a value that always falls between -1 and $+1$.

- r close to $+1$: strong positive linear relationship.
- r close to -1 : strong negative linear relationship.
- r close to 0 : weak or no linear relationship.

Because correlation is always on the same fixed scale regardless of units, it is far easier to compare across different pairs of variables than raw covariance.

Correlation does not imply causation. Two variables can be strongly correlated without one causing the other — both might be driven by a third, unobserved factor, or the relationship could be coincidental. Correlation only describes an observed statistical association, never a causal mechanism.

3.6 Python Implementation

```
df["feature"].mean()
df["feature"].var()
df["feature"].std()

df["feature1"].cov(df["feature2"])
df["feature1"].corr(df["feature2"])
```

Each of these Pandas methods returns a single number: the mean, variance, and standard deviation of one column, and the covariance or correlation between two columns. Inspecting these values during exploratory analysis helps identify which features vary meaningfully, which features are related to each other (potential redundancy), and which features might be related to the target variable.

Important Distinction

These statistics are primarily tools for **understanding and exploring** the dataset. Logistic regression, as implemented in libraries like Scikit-learn, does not require you to manually compute the mean, variance, covariance, or correlation before training — the algorithm learns directly from the raw (or scaled) feature values. You compute these statistics to build intuition about your data and to catch problems early, not because the model's training procedure depends on them.

4 Why Classification Needs Logistic Regression

Consider a simple binary classification problem: predicting whether a student passes an exam, based on the number of hours they studied.

$$y = \begin{cases} 0 & \text{Fail} \\ 1 & \text{Pass} \end{cases} \quad \text{Feature: Hours studied}$$

4.1 Why Not Just Use Linear Regression?

A natural first instinct is to reuse ordinary linear regression:

$$y = b_0 + b_1x$$

Here x is hours studied, and the model tries to fit a straight line through the data points, where y is treated as a numeric target of 0 or 1.

The problem is immediate: a straight line is not bounded. Depending on the input, this equation can output values such as:

$$-0.5, \quad 0.3, \quad 1.2, \quad 2.4$$

But a **probability** must always satisfy:

$$0 \leq p \leq 1$$

A predicted value of -0.5 or 2.4 has no meaningful interpretation as “the probability of passing.” Linear regression has no mechanism to keep its output confined to a valid probability range.

4.2 Why Not Just Clip the Output?

A tempting shortcut is to force the linear regression output into the $[0, 1]$ range — for example, clipping anything below 0 up to 0, and anything above 1 down to 1. This is not a good solution for two reasons:

- The clipped region becomes completely flat, meaning the model treats a student who studied 20 hours identically to one who studied 200 hours, discarding real information.
- The underlying line is still fit using squared-error assumptions that were never designed to model probabilities, so the relationship between x and the resulting probability remains awkward and poorly calibrated, especially near the extremes.

Clipping treats the symptom (out-of-range values) without fixing the underlying cause (a model shape that isn’t suited to representing probabilities in the first place).

4.3 Introducing Logistic Regression

Logistic regression solves this problem by design. Instead of directly outputting a raw linear score as the prediction, it takes that score and passes it through a mathematical function that *always* produces a value between 0 and 1.

Core Idea

Logistic Regression converts a linear combination of features into a probability using the **logistic (sigmoid) function**. The linear part still looks similar to linear regression, but its output is transformed before it is interpreted as a prediction.

The next chapter works through this transformation — the sigmoid function — in full mathematical detail.

5 Logistic Regression Mathematics

This chapter builds the core mathematics of logistic regression step by step.

5.1 The Linear Score

Every logistic regression model begins with a familiar linear combination of features:

$$z = b_0 + b_1x_1 + b_2x_2 + \cdots + b_px_p$$

- b_0 — the **intercept**, the value of z when all features equal zero.
- $b_1, \dots, b_p$ — the **model coefficients**, one per feature, learned from the training data.
- $x_1, \dots, x_p$ — the **features** (inputs) for a given example.
- z — the resulting **linear score**, which can be any real number, positive or negative.

z on its own is not yet a probability — it is an unbounded real number. The next step converts it into one.

5.2 The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function takes any real number z and squashes it into the range $(0, 1)$. The full process works as follows:

1. Each feature x_j is multiplied by its corresponding coefficient b_j .
2. These weighted features are added together, along with the intercept b_0 .
3. This sum produces the linear score z .
4. The sigmoid function converts z into a value strictly between 0 and 1.
5. That resulting value is interpreted as the **probability of the positive class** (i.e. class 1).

Worked Example

Suppose that for a particular example, the linear combination of features evaluates to:

$$z = 2$$

Applying the sigmoid function:

$$p = \frac{1}{1 + e^{-2}} \approx 0.881$$

This means the model estimates an 88.1% probability that this example belongs to class 1 (the positive class). Notice that this is a genuine probability — a single number between 0 and 1 that can be directly interpreted and compared, unlike the raw output of plain linear regression.

5.3 Sigmoid Behavior

The sigmoid function has three especially important behaviors, all visible in Figure 1:

As z approaches...	The probability approaches...
$z \rightarrow -\infty$	0
$z = 0$	0.5
$z \rightarrow +\infty$	1

Because the function rises smoothly from near 0, accelerates through the middle, and flattens out near 1, it produces the characteristic **S-shaped curve** shown below.

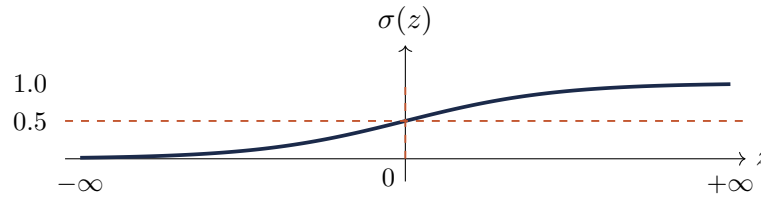


Figure 1: The sigmoid function: an S-shaped curve mapping any real number to the range (0, 1).

5.4 From Probability to Class

A logistic regression model outputs a probability, but a classification task ultimately needs a discrete class prediction. This is done using a **threshold**, most commonly 0.5:

$$\begin{aligned} p \geq 0.5 &\Rightarrow \text{Class 1} \\ p < 0.5 &\Rightarrow \text{Class 0} \end{aligned}$$

0.5 is only the **default** threshold, not a mathematical requirement. In applications where false negatives or false positives carry very different costs (for example, disease screening or fraud detection), it is common and often necessary to choose a different threshold to shift the balance between precision and recall. This idea is revisited in Section 9.

6 Log-Odds, Coefficients, and How the Model Learns

6.1 Odds and Log-Odds

Before connecting the sigmoid back to the linear score, it helps to define **odds**:

$$\text{Odds} = \frac{p}{1-p}$$

Odds express the same information as a probability, but as a ratio of “chance of success” to “chance of failure.” A probability of 0.8 corresponds to odds of $0.8/0.2 = 4$, meaning success is four times as likely as failure.

Taking the natural logarithm of the odds gives the **log-odds**, also called the **logit**:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

6.2 The Key Relationship

The entire mathematical foundation of logistic regression rests on one connection: the log-odds of the probability is exactly equal to the linear combination of features.

$$\ln\left(\frac{p}{1-p}\right) = b_0 + b_1x_1 + \cdots + b_px_p$$

In other words: the sigmoid function and the logit function are inverses of each other. The sigmoid takes a linear score z and turns it into a probability; the logit takes that probability and recovers the original linear score. This is precisely why logistic regression is called “logistic” — it directly models the log-odds as a linear function of the features, and the resulting probability is simply a consequence of that choice.

6.3 Interpreting Coefficients

Because the coefficients b_j act directly on the log-odds scale (not the probability scale), their interpretation must be handled carefully:

- A **positive coefficient** means that as the corresponding feature increases, the log-odds of the positive class increase — the model becomes more confident in class 1.
- A **negative coefficient** means that as the feature increases, the log-odds decrease — the model shifts toward class 0.
- A **larger absolute coefficient** generally indicates a stronger effect on the log-odds, but this comparison is only meaningful when features are on comparable scales, and can be affected by correlations between features and other modeling choices.

A logistic regression coefficient is **not** directly a percentage increase in probability. Because the sigmoid curve is non-linear, the same one-unit increase in a feature produces a different-sized change in probability depending on where you are on the curve (a change near $p = 0.5$ moves probability much more than the same change near $p = 0.98$). Coefficients operate on the log-odds scale, not the probability scale.

6.4 Odds Ratio

To make coefficients more interpretable, they are often converted into an **odds ratio**:

$$OR = e^{\beta_j}$$

Worked Example

Suppose a feature’s coefficient is:

$$\beta_j = 0.693$$

Then:

$$e^{0.693} \approx 2$$

This means that a one-unit increase in that feature multiplies the **odds** of the positive class by approximately 2, holding all other features constant. This is a much more intuitive way to communicate a coefficient’s effect than the raw log-odds value, and it is commonly reported alongside model coefficients in practice.

6.5 How Logistic Regression Learns

Fitting a logistic regression model means finding the coefficients $b_0, b_1, \dots, b_p$ that make the model’s predicted probabilities match the actual observed classes as closely as possible. This requires a way to measure “how wrong” the model’s predictions are, called a **loss function**.

Binary Cross-Entropy (Log Loss)

$$L = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right]$$

- y_i — the actual class (0 or 1) of example i .
- p_i — the model's predicted probability that example i belongs to class 1.

For each example, only one of the two terms inside the brackets is “active” (since either $y_i = 0$ or $1 - y_i = 0$). When the true label is 1, the loss depends on $\log(p_i)$: if the model predicted a probability close to 1, this term is close to zero (low loss); if the model confidently predicted close to 0, $\log(p_i)$ becomes a large negative number, producing a very large loss. The symmetric logic applies when the true label is 0.

This is why log loss is described as heavily penalizing **confident incorrect predictions**: being wrong is costly, but being wrong *and confident* is far more costly than being wrong while uncertain.

Optimization

Training the model means searching for the coefficients that make this average loss as small as possible across the entire training set. This search is typically performed using **gradient-based optimization**: starting from some initial coefficients, the algorithm repeatedly computes the direction in which the loss decreases fastest and takes a small step in that direction, gradually converging on coefficients that fit the data well. Scikit-learn handles this optimization internally — as a practitioner, you generally do not need to implement the gradient calculations yourself, but understanding that this iterative minimization is happening “under the hood” when you call `.fit()` makes the whole process far less mysterious.

7 Implementing Logistic Regression in Python

With the mathematics established, this chapter walks through the simplest complete implementation using Scikit-learn.

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("dataset.csv")

X = df[["feature1", "feature2"]]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
y_prob = model.predict_proba(X_test)
```

7.1 Line-by-Line Explanation

Why import Pandas?

Pandas provides the `DataFrame` structure used to load, inspect, and manipulate tabular data — exactly the format most Kaggle datasets come in (CSV files with rows and named columns).

Why import `train_test_split()`?

A model that is only ever evaluated on the same data it was trained on tells you nothing about how it will perform on new, unseen data. `train_test_split()` sets aside a portion of the data purely for evaluation, so that the reported performance reflects genuine generalization.

Why import `LogisticRegression()`?

This is the Scikit-learn class that implements the logistic regression algorithm described in Sections 5 and 6 — it handles the linear score, the sigmoid transformation, and the optimization of coefficients internally.

Why separate `X` and `y`?

Scikit-learn expects features and targets as separate objects: `X` holds the input columns the model will learn from, and `y` holds the single column of correct answers the model is trying to predict.

Why `test_size=0.2`?

This reserves approximately 20% of the data for testing and uses the remaining 80% for training — a common and reasonable split for small-to-medium datasets, balancing enough training data with a large enough test set to trust the evaluation.

Why `random_state`?

Setting a fixed `random_state` makes the train/test split reproducible: running the code again produces the exact same split, which is important for debugging and for fair comparisons between different models or settings.

Why `stratify=y`?

Stratifying by `y` ensures the proportion of each class is preserved in both the training and testing sets. Without this, a random split could accidentally place too many (or too few) examples of the minority class into the test set, especially in imbalanced datasets, producing a misleading evaluation.

What does `fit()` do?

`model.fit(X_train, y_train)` runs the training process: it searches for the coefficients $b_0, \dots, b_p$ that minimize the log loss on the training data, as described in Section 6. After this call, the model object holds the learned coefficients.

What does `predict()` do?

`model.predict(X_test)` applies the default 0.5 threshold to the model's predicted probabilities and returns the final predicted **class labels** (e.g. 0 or 1) for each row in `X_test`.

What does `predict_proba()` do?

`model.predict_proba(X_test)` returns the underlying **probabilities** for each class, before any threshold is applied.

7.2 Predict vs Predict-Prob

Method	Returns	Example output
<code>predict()</code>	Final class labels	[0, 1, 1, 0]
<code>predict_proba()</code>	Probability of each class	[0.82, 0.18] (for one row: [P(class 0), P(class 1)])

The key difference: `predict()` gives you a final decision, while `predict_proba()` gives you the underlying confidence behind that decision. A row with probabilities [0.82, 0.18] means the model is 82% confident in class 0 and 18% confident in class 1 — since $0.18 < 0.5$, `predict()` would return class 0 for this row. Having access to the raw probabilities is essential for tasks like adjusting the decision threshold, ranking predictions by confidence, or computing metrics such as ROC-AUC.

8 Binary and Multiclass Logistic Regression

8.1 Binary Logistic Regression

In the binary case, already covered in detail, the target has two classes, for example:

$$0 \rightarrow \text{No Churn} \quad 1 \rightarrow \text{Churn}$$

The sigmoid function produces a single probability: the probability of the **positive class** (class 1). The probability of the negative class is simply 1 minus that value. This is the simplest and most common use case for logistic regression.

8.2 Multiclass Classification

Many real problems involve more than two categories. Consider the classic example of iris flower classification:

$$0 \rightarrow \text{Setosa} \quad 1 \rightarrow \text{Versicolor} \quad 2 \rightarrow \text{Virginica}$$

Here, a single sigmoid output (one probability) is no longer sufficient — the model needs to express a probability for *each* of the three classes, and those probabilities must sum to 1. This is achieved with the **softmax function**, a generalization of the sigmoid to multiple classes.

$$P(y = k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- Each class k gets its own linear score z_k , computed from its own set of coefficients.
- Softmax exponentiates each score and divides by the sum of all exponentiated scores, converting the raw scores into a valid probability distribution.
- The resulting probabilities are all positive and **sum to exactly 1** across the classes.

Worked Example

Class	Predicted Probability
Setosa	0.10
Versicolor	0.70
Virginica	0.20

The predicted class is simply the one with the highest probability:

Prediction = **Versicolor**

8.3 One-vs-Rest vs Multinomial

Scikit-learn offers two broad strategies for extending logistic regression to more than two classes.

One-vs-Rest (OvR)

The multiclass problem is broken down into several independent binary problems. For 3 classes, this means training three separate binary classifiers:

Class A vs Rest Class B vs Rest Class C vs Rest

Each classifier answers “is this example Class X, or is it something else?” At prediction time, the class whose binary classifier is most confident wins.

Multinomial

Rather than training separate binary models, a single multiclass model directly models the probability of each class simultaneously using the softmax function shown above. This is generally considered the more statistically natural approach for multiclass logistic regression when the classes are mutually exclusive.

Which One to Use

Modern versions of Scikit-learn automatically choose a sensible default multiclass strategy based on the solver used, and multinomial handling is generally preferred when available. As a practitioner, it is enough to know that both strategies exist and that they can produce slightly different results; you do not need to master the internal optimization differences to use logistic regression effectively for multiclass problems.

9 Evaluating Classification Models

A model is only useful if we can measure how well it actually performs. This chapter introduces the confusion matrix and the metrics derived from it.

9.1 The Confusion Matrix

The confusion matrix breaks predictions down by comparing what the model predicted against what actually happened.

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Using fraud detection (Positive = Fraud) as a practical example:

- **TP (True Positive)** — an actual fraud case that the model correctly flagged as fraud.
- **FN (False Negative)** — an actual fraud case that the model missed, predicting it as not fraud. This is a dangerous error: real fraud goes undetected.
- **FP (False Positive)** — a legitimate transaction that the model incorrectly flagged as fraud. This causes unnecessary friction for genuine customers.
- **TN (True Negative)** — a legitimate transaction correctly identified as not fraud.

Every classification metric below is built directly from these four counts.

9.2 Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy measures the overall proportion of predictions the model got right. It is simple and intuitive, but it is only an appropriate primary metric when the classes are roughly balanced — with imbalanced data, it can be dangerously misleading (see Section 10).

9.3 Precision

$$Precision = \frac{TP}{TP + FP}$$

“Of all cases predicted positive, how many were actually positive?” High precision means that when the model raises an alarm, it is usually right — few false alarms.

9.4 Recall

$$Recall = \frac{TP}{TP + FN}$$

“Of all actual positive cases, how many did the model correctly identify?” High recall means the model catches most of the real positive cases, at the potential cost of more false alarms.

9.5 Specificity

$$Specificity = \frac{TN}{TN + FP}$$

Specificity measures how well the model identifies actual negatives correctly — it is the negative-class counterpart to recall, answering “of all actual negative cases, how many did the model correctly label as negative?”

9.6 F1 Score

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall}$$

F1 is the **harmonic mean** of precision and recall. The harmonic mean is used (rather than a simple average) because it penalizes extreme imbalances between the two: a model with precision 1.0 and recall 0.01 would have a high arithmetic average but a very low F1 score, correctly reflecting that the model is not actually useful. F1 is a good single-number summary when you need to balance precision and recall together.

9.7 Precision vs Recall: Choosing What Matters

Scenario	Why it matters
Spam filtering	A false positive (blocking a real email) can be very costly to the user, so precision is often prioritized.
Disease detection	A false negative (missing an actual illness) can be life-threatening, so recall is often prioritized.

There is no universally “correct” metric — the right choice depends entirely on the relative real-world cost of false positives versus false negatives in the specific application.

9.8 ROC-AUC

The **ROC curve** (Receiver Operating Characteristic) plots the True Positive Rate against the False Positive Rate across every possible classification threshold, not just 0.5.

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

Note that TPR is identical to recall. As the decision threshold is lowered from 1 to 0, both TPR and FPR increase, tracing out a curve. The **AUC** (Area Under the Curve) condenses this entire curve into a single number between 0 and 1: an AUC of 0.5 indicates performance no better than random guessing, while an AUC of 1.0 indicates perfect separation between classes. ROC-AUC is valuable because it evaluates the model’s ability to **rank** positive examples above negative ones, independent of any single chosen threshold.

On highly imbalanced datasets, ROC-AUC can look deceptively good because the False Positive Rate denominator ($FP + TN$) is dominated by the large number of true negatives. In these situations, a **Precision-Recall curve** is often more informative, since it focuses entirely on how the model handles the positive (usually minority) class.

9.9 Evaluating Models in Python

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score
)

print(confusion_matrix(y_test, y_pred))
print(accuracy_score(y_test, y_pred))
print(precision_score(y_test, y_pred))
print(recall_score(y_test, y_pred))
print(f1_score(y_test, y_pred))
print(roc_auc_score(y_test, y_prob[:, 1]))
```

Each function computes exactly the metric its name suggests, using `y_test` (the true labels) and `y_pred` (the predicted labels) as inputs. The one exception is `roc_auc_score`, which needs **probabilities** rather than hard class labels, because it evaluates performance across every possible threshold. `y_prob[:, 1]` selects the second column of the probability array — the probability of class 1 — since that is the convention for binary ROC-AUC in Scikit-learn.

Multiclass Averaging

For multiclass problems, metrics like precision, recall, and F1 must be combined across classes, using one of three averaging strategies:

- **Macro averaging** — computes the metric separately for each class, then takes the unweighted average, treating every class as equally important regardless of size.
- **Weighted averaging** — computes the metric per class, then averages using each class's frequency as a weight, giving larger classes more influence.
- **Micro averaging** — aggregates the total TP, FP, and FN counts across all classes first, then computes the metric once from those totals, effectively weighting every individual prediction equally.

10 Class Imbalance, Overfitting, Regularization and Preprocessing

This chapter combines several practical concerns that directly determine whether a logistic regression model will actually work well in the real world.

10.1 Class Imbalance

Consider a fraud detection dataset where:

95% → Not Fraud 5% → Fraud

A model that simply predicts “Not Fraud” for every single transaction, without learning anything at all, would achieve 95% accuracy. This makes the model look excellent by the accuracy metric, while being completely useless for its actual purpose — it would never catch a single fraudulent transaction.

Why accuracy can fail: accuracy treats every correct prediction equally, regardless of class. On imbalanced data, the majority class dominates the metric, hiding the model's complete failure on the minority class — which is usually the class we actually care about most.

This is exactly why **precision**, **recall**, and **F1** (Section 9) are essential on imbalanced datasets: they specifically expose how the model handles the minority class, which accuracy conceals.

Class Weights

One practical fix is to tell the model to pay more attention to the minority class during training:

```
LogisticRegression(class_weight="balanced")
```

Conceptually, this increases the penalty (loss contribution) for misclassifying minority-class examples, and decreases the penalty for majority-class errors, proportional to how rare each class is. This encourages the model to take the minority class seriously rather than ignoring it in favor of overall accuracy.

10.2 Overfitting and Underfitting

Underfitting

A model **underfits** when it is too simple to capture the real patterns in the data, resulting in poor performance even on the training data it was fit to.

Overfitting

A model **overfits** when it learns patterns that are specific to the quirks and noise of the training data, rather than the true underlying relationship. It performs very well on training data but poorly on new, unseen data.

Situation	Train Performance	Test Performance
Underfitting	Poor	Poor
Good generalization	Good	Good
Overfitting	Very Good	Poor

These situations connect to the classic **bias-variance** trade-off: underfitting corresponds to high *bias* (the model's assumptions are too simplistic to capture reality), while overfitting corresponds to high *variance* (the model is too sensitive to the specific training sample, and would produce very different results on a different training sample). Good models aim to balance the two.

10.3 Regularization

Regularization discourages a model from fitting the training data *too* closely, by adding a penalty for large coefficients directly into the loss function being minimized.

L1 Regularization (Lasso)

$$Loss + \lambda \sum |\beta_j|$$

L1 regularization adds a penalty proportional to the **absolute value** of each coefficient. This has a distinctive effect: it can shrink some coefficients all the way to exactly zero, effectively removing those features from the model. This makes L1 useful for implicit **feature selection** when many features may be irrelevant.

L2 Regularization (Ridge)

$$Loss + \lambda \sum \beta_j^2$$

L2 regularization adds a penalty proportional to the **squared value** of each coefficient. It shrinks all coefficients toward zero, but rarely forces any of them to be exactly zero. This tends to produce more stable models when features are correlated with each other.

Scikit-learn's C Parameter

```
LogisticRegression(C=1.0)
```

Scikit-learn controls regularization strength through the parameter **C**, which is the **inverse** of the regularization strength λ :

Remember This

Small C → stronger regularization (coefficients pushed harder toward zero)
Large C → weaker regularization (coefficients allowed to grow more freely)

This is the inverse of what many learners initially expect, because C is defined as $1/\lambda$: a small C corresponds to a large effective λ (strong penalty), while a large C corresponds to a small effective λ (weak penalty).

10.4 Feature Scaling

Consider two features on very different scales:

$$\text{Age} = 25 \qquad \text{Income} = 800,000$$

Because logistic regression's optimization process and its regularization penalty both operate directly on the coefficient values, features with much larger raw magnitudes can dominate the loss and the penalty term simply due to their scale, not because they are more predictive. Scaling puts all features on a comparable footing.

```
from sklearn.preprocessing import StandardScaler
```

`StandardScaler` transforms each feature to have a mean of 0 and a standard deviation of 1, so that a one-unit change means the same thing (one standard deviation) across every feature.

To apply this correctly and consistently, it is combined with the model inside a **Pipeline**:

```
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
])
```

The `Pipeline` chains preprocessing steps and the final model into a single object. Calling `model.fit(X_train, y_train)` first fits and applies the scaler to the training data, then trains the logistic regression on the scaled result. Calling `model.predict(X_test)` automatically applies the *same* scaling (learned from training data) to the test data before making predictions — ensuring consistent, correctly-ordered preprocessing every time.

Feature scaling is especially important for logistic regression because both the gradient-based optimization and any L1/L2 regularization penalty are directly sensitive to the numerical scale of each feature.

10.5 Data Leakage

Definition

Data leakage occurs when information from outside the training process improperly influences model training — most often when information from the test set (or the future) leaks into the training process, making evaluation results overly optimistic and unreliable.

Common sources of leakage include:

- **Scaling** — fitting a `StandardScaler` on the *entire* dataset (including test data) before splitting, so the scaler's mean and standard deviation are influenced by test data it should never have seen.
- **Imputation** — filling missing values using statistics (like the mean) calculated from the full dataset rather than from training data alone.
- **Feature selection** — choosing which features to keep based on their relationship with the target across the whole dataset, before the train/test split has been made.

Using a `Pipeline`, as shown above, helps prevent this class of mistakes: because the scaler (and any other preprocessing step) is fit only within the training fold and then only *applied* to the test fold, test data is never used to influence how the training data or the model itself is prepared.

11 Complete Kaggle-Style Classification Project

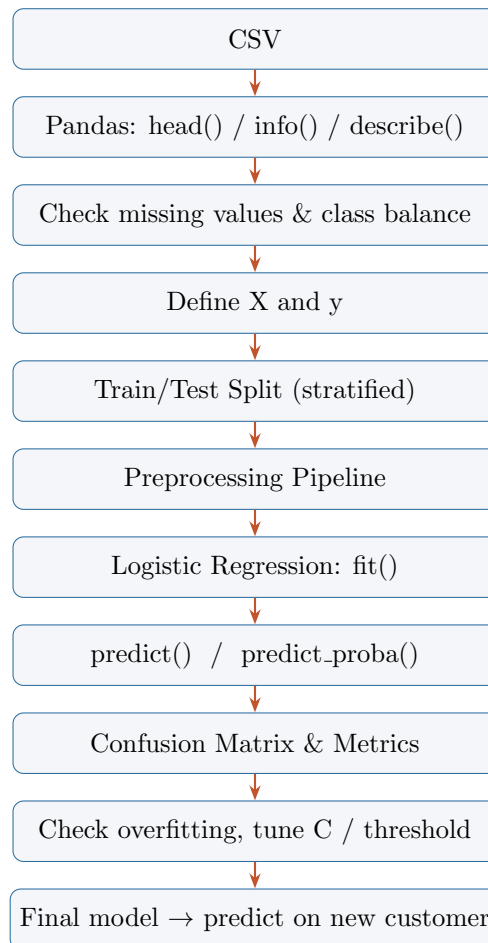
This chapter ties everything together into a single, realistic, end-to-end project: predicting **customer churn**.

11.1 The Project

Features: Age, Tenure, Monthly Charges, Contract Type, Payment Method, Number of Services.

Target: Churn — 0 = No, 1 = Yes.

11.2 The Workflow



11.3 Step 1 – Load and Inspect

```
import pandas as pd

df = pd.read_csv("churn.csv")

df.head()
df.info()
df.describe()

df.isnull().sum()
df["Churn"].value_counts(normalize=True)
```

The last line reveals the class balance — for example, discovering that only 15% of customers churned tells us immediately that accuracy alone will not be a reliable evaluation metric (Section 10).

11.4 Step 2 – Define Features and Target, Then Split

```
X = df.drop(columns=["Churn"])
y = df["Churn"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

11.5 Step 3 – Preprocessing Pipeline

This dataset has both numeric columns (Age, Tenure, Monthly Charges, Number of Services) and categorical columns (Contract Type, Payment Method). Each type needs different preprocessing, which is handled cleanly with a ColumnTransformer.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_features = ["Age", "Tenure", "MonthlyCharges", "NumServices"]
categorical_features = ["ContractType", "PaymentMethod"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

- `SimpleImputer` fills missing values (median for numeric columns, most frequent category for categorical columns) so that no row needs to be dropped just because of a small gap.
- `OneHotEncoder` converts categorical text columns (like “Month-to-month” or “Credit Card”) into numeric 0/1 indicator columns, since logistic regression requires purely numeric input. `handle_unknown="ignore"` prevents errors if the test set contains a category not seen during training.
- `ColumnTransformer` applies the numeric pipeline to numeric columns and the categorical pipeline to categorical columns, then combines the results into a single feature matrix.

11.6 Step 4 – Full Model Pipeline and Training

```
from sklearn.linear_model import LogisticRegression

model = Pipeline([
```

```

    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression(class_weight="balanced", C=1.0))
])

model.fit(X_train, y_train)

```

Wrapping the `ColumnTransformer` and the `LogisticRegression` classifier together inside one outer Pipeline ensures all preprocessing is fit only on training data and consistently reapplied to test data and any future new data, preventing the data leakage issues discussed in Section 10.

11.7 Step 5 – Predict and Evaluate

```

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)

from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    precision_score, recall_score,
    f1_score, roc_auc_score
)

print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_prob[:, 1]))

```

11.8 Step 6 – Interpret Errors and Check Overfitting

Compare training and test performance using the same metrics:

```

train_acc = model.score(X_train, y_train)
test_acc = model.score(X_test, y_test)

print("Train accuracy:", train_acc)
print("Test accuracy:", test_acc)

```

If training accuracy is much higher than test accuracy, this signals overfitting (Section 10), and reducing `C` (increasing regularization strength) is a reasonable next step. If both are low, this signals underfitting, and adding more informative features or reducing regularization may help.

11.9 Step 7 – Tune Threshold if Necessary

If recall on the churn class is too low (the business cares more about catching potential churners than avoiding false alarms), the classification threshold can be lowered below the default 0.5:

```

threshold = 0.35
y_pred_adjusted = (y_prob[:, 1] >= threshold).astype(int)

```

This trades some precision for higher recall — a reasonable choice for a churn-prevention campaign where the cost of missing a churning (a lost customer) may outweigh the cost of a false alarm (a small retention offer sent unnecessarily).

11.10 Step 8 – Predict for a New Customer

```

new_customer = pd.DataFrame([

```

```

    "Age": 42,
    "Tenure": 5,
    "MonthlyCharges": 89.5,
    "NumServices": 3,
    "ContractType": "Month-to-month",
    "PaymentMethod": "Electronic check"
  })

prediction = model.predict(new_customer)
probability = model.predict_proba(new_customer)

print("Predicted class:", prediction[0])
print("Churn probability:", probability[0][1])

```

Because `model` is a full Pipeline, calling `.predict()` on a brand-new, raw, unprocessed customer record automatically applies the exact same imputation, scaling, and encoding used during training, before making the final prediction — exactly how a model is used in a real, deployed setting.

12 Binary vs Multiclass + Logistic Regression Decision Guide

This chapter collects the key comparisons from across the guide into quick-reference tables.

12.1 Binary vs Multiclass

Aspect	Binary	Multiclass
Classes	2	3+
Example	Fraud / Not Fraud	Cat / Dog / Horse
Main probability idea	Sigmoid	Softmax / multiclass strategy (OvR or Multinomial)
Output	Single probability + class	Probability distribution across all classes

12.2 Logistic Regression vs Linear Regression

Aspect	Linear Regression	Logistic Regression
Problem type	Regression	Classification
Output	Continuous value	Probability / class
Core function	Linear equation	Sigmoid / softmax
Typical objective	Squared-error based (MSE)	Log loss (cross-entropy)
Example	House price prediction	Customer churn prediction

12.3 Evaluation Metric Guide

Situation	Important Metric
Balanced classes	Accuracy can be useful
False positives are costly	Precision
False negatives are costly	Recall
Need a balance of precision and recall	F1 Score
Strong class imbalance	Precision / Recall / F1
Ranking predictions by confidence	ROC-AUC

Keep in Mind

These tables are practical guidelines, not rigid rules. Real projects often require looking at several metrics together, understanding the specific business cost of different error types, and adjusting the decision threshold accordingly, rather than mechanically optimizing a single number.

13 Final Formula Sheet

A compact mathematical reference collecting every formula from this guide, with a brief reminder of what each symbol means.

13.1 Descriptive Statistics

Mean: $\bar{x} = \frac{1}{n} \sum x_i$ — average value; n = number of observations.

Variance: $Var(X) = \frac{1}{n} \sum (x_i - \bar{x})^2$ — average squared distance from the mean.

Standard Deviation: $\sigma = \sqrt{Var(X)}$ — spread, in original units.

Covariance: $Cov(X, Y) = \frac{1}{n} \sum (x_i - \bar{x})(y_i - \bar{y})$ — joint variability of two variables.

Correlation: $r = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$ — scaled covariance, always in $[-1, 1]$.

13.2 Logistic Regression Core

Linear Score: $z = b_0 + \sum_{j=1}^p b_j x_j$ — b_0 intercept, b_j coefficients, x_j features.

Sigmoid: $p = \frac{1}{1 + e^{-z}}$ — converts z into a probability in $(0, 1)$.

Odds: $Odds = \frac{p}{1-p}$ — ratio of success to failure probability.

Logit (log-odds): $logit(p) = \ln\left(\frac{p}{1-p}\right)$ — equals the linear score z .

Odds Ratio: $OR = e^{\beta_j}$ — multiplicative effect of a one-unit feature increase on the odds.

Log Loss: $L = -\frac{1}{n} \sum [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$ — y_i actual class, p_i predicted probability.

Softmax: $P(y = k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ — multiclass generalization of the sigmoid.

13.3 Regularization

L1 (Lasso): $Loss + \lambda \sum |\beta_j|$ — can shrink coefficients exactly to zero.
L2 (Ridge): $Loss + \lambda \sum \beta_j^2$ — shrinks coefficients smoothly toward zero.

13.4 Classification Evaluation Metrics

Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$
Precision: $\frac{TP}{TP + FP}$
Recall (TPR): $\frac{TP}{TP + FN}$
Specificity: $\frac{TN + FP}{TN + FP + TP}$
F1 Score: $2 \frac{Precision \times Recall}{Precision + Recall}$
False Positive Rate (FPR): $\frac{FP}{FP + TN}$

where TP = true positives, TN = true negatives, FP = false positives, FN = false negatives.

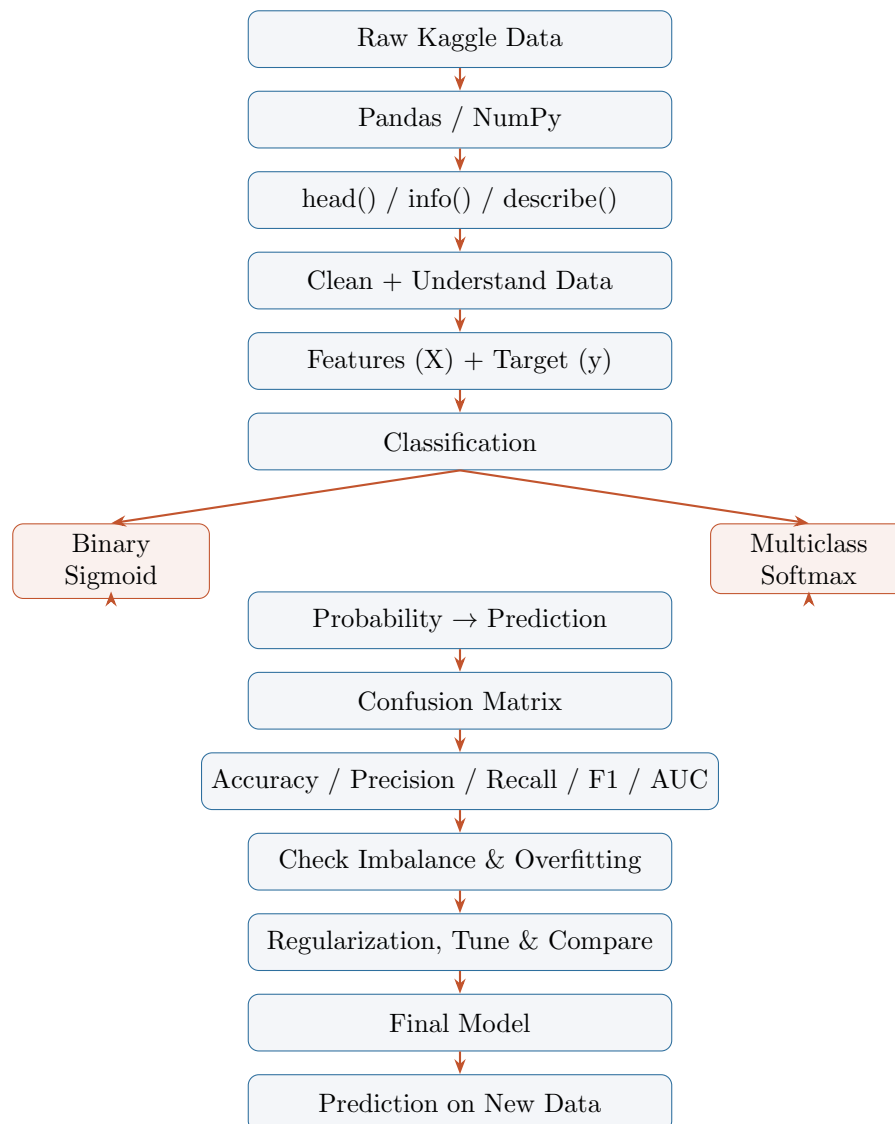
14 Final Python Cheat Sheet

Code	Purpose
<code>pd.read_csv()</code>	Load a dataset from a CSV file
<code>df.head()</code>	Preview the first rows of the data
<code>df.info()</code>	Inspect structure: rows, columns, types, and nulls
<code>df.describe()</code>	Statistical summary of numeric columns
<code>df.isnull()</code>	Detect missing values
<code>df.drop()</code>	Remove rows or columns
<code>df.fillna()</code>	Handle missing values
<code>train_test_split()</code>	Separate data into training and testing sets
<code>StandardScaler()</code>	Scale numerical features (mean 0, std 1)
<code>OneHotEncoder()</code>	Encode categorical features numerically
<code>Pipeline()</code>	Combine preprocessing steps and a model
<code>LogisticRegression()</code>	Create a logistic regression classifier
<code>fit()</code>	Train the model on training data
<code>predict()</code>	Predict class labels
<code>predict_proba()</code>	Predict class probabilities
<code>confusion_matrix()</code>	Analyze classification errors
<code>accuracy_score()</code>	Compute accuracy
<code>precision_score()</code>	Compute precision

Code	Purpose
<code>recall_score()</code>	Compute recall
<code>f1_score()</code>	Compute F1 score
<code>roc_auc_score()</code>	Compute ROC-AUC

15 Final Mental Model

The diagram below summarizes the entire pipeline covered in this guide, from raw data to a final deployed prediction.



In plain language: everything starts with raw data, which must first be inspected and cleaned before it means anything. Once features and a target are defined, the type of classification problem (binary or multiclass) determines whether the sigmoid or softmax function converts the model's internal linear score into a probability. That probability becomes a prediction, which is then rigorously evaluated using a confusion matrix and the metrics built from it — always with an eye on class imbalance and overfitting. Regularization and threshold tuning refine the model, and the result is a final model ready to generate predictions on brand-new, real-world data.

From raw data to a trustworthy prediction — that is the complete arc of applied logistic regression.